

프로젝트 결과보고서

기본 사항				
교과목명	시스템 프로그래밍		담당교수	김성우
프로젝트 주제	간단한 셸 프로그램 만들기			
팀구성원 (학번/성명)	20163335/곽지원	20163366/김예원	20161602/박경인	20163291/백소현
개발 환경	리눅스 및 GNU 프로그래밍 개발 도구 (gcc, make, gdb) 등			
프로젝트 주소	https://github.com/gold37/shell			

1. 프로젝트 개요

shell은 운영체제의 커널과 사용자의 사이를 연결시켜주는 역할을 하는 것으로 사용자가 명령을 내리면 그 명령을 해석하고 운영체제가 그에 따라 프로그램이 실행되는 역할을 한다. shell의 지시어를 받으면 운영체제는 하드웨어를 위한 지시어로 변경을 시켜준다. 이러한 점을 이용하여 리눅스 안에서 파일을 수정하거나 디렉토리 변경 및 수정, 통신 프로세스등에 대한 컴퓨터 운영체제의 구동이 가능한 것처럼 내가 직접 사용할 나만의 셸 프로그램을 C언어로 작성하면 손쉽게 운영체제 이용이 가능하도록 할 수 있다. 기존에 없던 새로운 프로그램의 소스를 구현하여 내가 원하는 명령을 수행 시킬 수 있게 생성하는 것이 가능하다. 시그널 처리와, 파이프 기능, 리눅스 기본 명령어 및 파일처리 명령어, 좀비 프로세스 구현, 파일 재 지향, 셸 내의 프로그램 종료 명령어를 구현하는 하나의 간단한 셸 프로그램을 만들 수 있다.

2. 설계 구성 요소

목표/기준 설정	합 성	분 석	제 작	시 험	평 가	결과 도출	기 타
	√	√	√	√			

목표 설정	셸 프로그램을 사용하기 위하여 간단하게 프로그램을 만들도록 목표를 정했다.
합 성	csh, bash등에서처럼 셸 명령의 마지막에 '&'를 입력하여 백그라운드를 실행하도록 하는 기능과 파이프() 기능을 이용하여 새로운 SW를 재생산하기 용이하게 프로그램을 작성할 수 있었다.

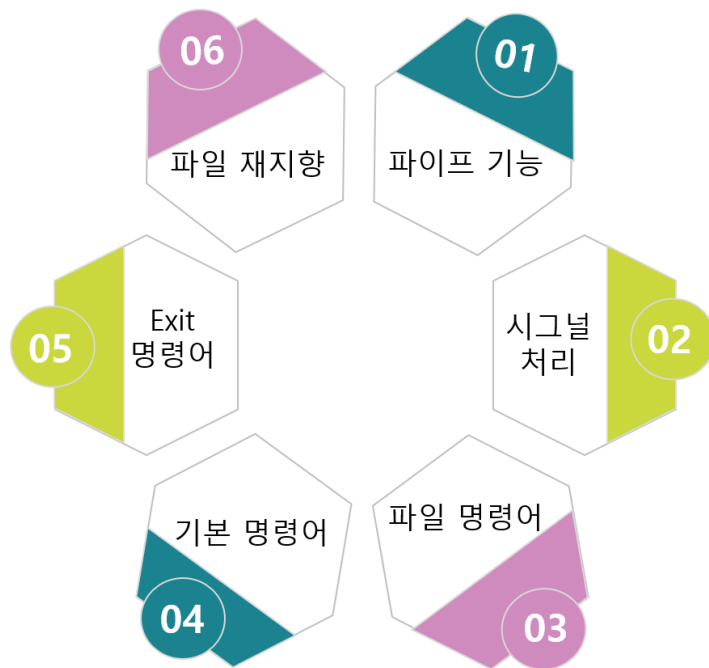
분 석	운영체제의 커널과 사용자 사이를 이어주는 역할을 하는 쉘 프로그램을 통해 시그널 처리, 파이프 기능, 리눅스 기본 명령어 및 파일처리 명령어, 좀비프로세스 구현, 파일 재지향, 쉘 내 프로그램 종료 명령어등을 구현할 수 있어야 한다.
제 작	일정 계획 Task List (from top to bottom): - 프로그램 설계 - 프로그램 개요 작성 - 설계 구성 요소 작성 - 역할 분담 - 일정 관리 계획 작성 - 시그널 처리 - 파이프 기능 - 기본 명령어 및 파일처리 명령어 - 좀비프로세스 - 파일 재지향 - 셸 내 프로그램 종료 명령어 - 셸 프로그램 통합 - 셸 프로그램 구현 - 프로젝트 결과 보고서 작성 Timeline: 2018-11-12, 2018-11-14, 2018-11-16, 2018-11-18, 2018-11-20, 2018-11-22 Legend: ■ 기간
시 험	완성된 설계 프로젝트에 대한 모의실험을 쉘 프로그램을 통해 수행할 수 있다.

3. 설계 제한 요소

경제성	안정성	신뢰성	미 학	윤리성	사회적 영향	생산성/내구성	산업표준	기타
√	√					√		

경제성	구현한 소스코드 분량은 10000라인 미만정도로 프로그램의 분량은 적절하다. 불필요한 소스코드의 부분을 제거하여 효율적인 쉘 프로그램을 작성하는 것을 원칙으로 하였다.
안정성	셸 프로그램을 실행 하였을 때, 구현한 소스에 대한 명령어들이 지속적으로 외부환경에 대한 변경없이 잘 동작함을 확인할 수 있다.
생산성	설계 프로젝트에 필요한 기능들(시그널 처리와, 파이프 기능, 리눅스 기본 명령어 및 파일처리 명령어, 좀비프로세스 구현, 파일 재지향, 쉘 내 프로그램 종료 명령어)이 대부분 실행이 되지만, 몇몇 기능에 있어서는 동작하지 않는 부분이 있다. 구현한 기능들로 사용자의 요구사항을 만족하는 운영체제 이용이 가능하다.

4. 설계 내용



5. 프로젝트 결과

5.1 프로젝트 구현 내용

11-1. "exit"를 치면 프로그램을 끝내도록 프로그램을 수정하시오.

[소스코드 및 설명]

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include "common.h"
```

////////////////////////////////////

```
// 표준 입/출력을 이용한 리다이렉트 함수
```

```
////////////////////////////////////
```

```
void redirect(int sourcefd, char *sourcefile, int destfd, char *destfile, BOOLEAN append, BOOLEAN  
backgrnd)
```

```
{
```

```
    int flags, fd;
```

```
    // 열려있는 소스 파일디스크립터가 없고 백그라운드 실행일 경우
```

```
    if (sourcefd == 0 && backgrnd)
```

```
    {
```

```
        strcpy(sourcefile, "/dev/null");
```

```
        sourcefd = BADFD;
```

```
    }
```

```
    // 소스 파일 디스크립터가 있을 경우
```

```
    if (sourcefd != 0)
```

```
    {
```

```
        // 표준입력을 닫는다.
```

```
        if (close(0) == ERROR)
```

```
        {
```

```
            syserr("close");
```

```
        }
```

```
        if (sourcefd > 0)
```

```
        {
```

```
            // 표준입력을 재정의한다.
```

```
            if (dup(sourcefd) != 0)
```

```
            {
```

```
                fatal("dup");
```

```
            }
```

```

    }

    // 소스파일을 연다.
    else if (open(sourcefile, O_RDONLY, 0) == ERROR)
    {
        fprintf(stderr, "Cannot open %s\n", sourcefile);
        exit(0);
    }
}

// 출력 파일 디스크립터가 있다면
if (destfd != 1)
{
    // 표준 출력을 닫는다.
    if (close(1) == ERROR)
    {
        syserr("close");
    }

    // 표준 출력을 재정의
    if (destfd > 1)
    {
        if (dup(destfd) != 1)
        {
            fatal("dup");
        }
    }
}

else

```

```

{
    // 파일이 없을 경우에 생성하고, 쓰기도 가능하게 파일을 설정
    flags = O_WRONLY | O_CREAT;

    // 추가 모드가 아닐 경우 파일을 비우는 옵션도 추가
    if (!append)
    {
        flags |= O_TRUNC;
    }

    // 파일을 연다.
    if (open(destfile, flags, 0666) == ERROR)
    {
        fprintf(stderr, "Cannot create %s\n", destfile);
        exit(0);
    }

    // 추가 모드라면 가장 끝을 찾는다.
    if (append)
    {
        if (lseek(1, 0L, 2) == ERROR) syserr("lseek");
    }
}

// 파일디스크립터를 닫는다.
for (fd =3; fd < MAXFD; fd++)
{
    close(fd);
}

```

```

    }

    return;
}

```

[실행결과]

```

jiwon@ubuntu:~/lab3$ ./minish
>> exit
jiwon@ubuntu:~/lab3$

```

11-2. csh, bash 등에서처럼 쉘 명령의 마지막에 ‘&’을 입력하면 백그라운드로 실행되도록 프로그램을 수정하십시오.

[소스코드 및 설명]

```

#include <stdio.h>

#include <stdlib.h>

int main(void)
{
    pid_t pid;

    if((pid = fork()) < 0) {
        perror("fork");
        exit(1);
    }

    else if (pid == 0)
        exit(0);

    sleep(4);

    system("ps -o pid, ppid, state, tty, command");

    exit(0);
}

```

[실행 결과]

```
gyeongin@ubuntu:~$ ./zombie.c&
[1] 11851
gyeongin@ubuntu:~$ bash: ./zombie.c: Permission denied
[1]+  Exit 126                  ./zombie.c
```

11-3. csh, bash 등에서처럼 인터럽트키 (SIGINT: Ctrl-C, SIGQUIT: Ctrl-Z)가 동작하도록 프로그램을 수정하시오.

[소스코드 및 설명]

```
/* handlesignal.c */

/* sigaction example */

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int signo);

main()
{
    struct sigaction act;
    int i = 0;

    act.sa_handler = handler;
    sigfillset(&(act.sa_mask));
    sigaction(SIGINT, &act, NULL);
    printf("SIGINT on\n");
    while(1) {
        sleep(1);
        printf("sleep for %d sec(s).\n", ++i);
    }
}

void handler(int signo)
{

```



```
printf("handler: signo=%d\n", signo);
}
```

[실행 결과]

```
yewonii@ubuntu:~$ gcc handlesignal.c -o handlesignal
handlesignal.c:6:1: warning: return type defaults to 'int' [-Wimplicit-int]
main()
^~~~~
yewonii@ubuntu:~$ ./handlesignal
SIGINT on
sleep for 1 sec(s).
sleep for 2 sec(s).
sleep for 3 sec(s).
sleep for 4 sec(s).
^Chandler: signo=2
sleep for 5 sec(s).
sleep for 6 sec(s).
sleep for 7 sec(s).
^Z
[2]+  Stopped                  ./handlesignal
```

11-4. 파일 재지향 (>, <) 기능이 가능하도록 프로그램을 수정하시오

[소스코드 및 설명]

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include "common.h"
```

```
////////////////////////////////////
```

```
// 표준 입/출력을 이용한 리다이렉트 함수
```

```
////////////////////////////////////
```

```
void redirect(int sourcefd, char *sourcefile, int destfd, char *destfile, BOOLEAN append, BOOLEAN
backgrnd)
```

```
{
```

```
    int flags, fd;
```

// 열려있는 소스 파일디스크립터가 없고 백그라운드 실행일 경우

```
if (sourcefd == 0 && backgrnd)
```

```
{
```

```
    strcpy(sourcefile, "/dev/null");
```

```
    sourcefd = BADFD;
```

```
}
```

// 소스 파일 디스크립터가 있을 경우

```
if (sourcefd != 0)
```

```
{
```

```
    // 표준입력을 닫는다.
```

```
    if (close(0) == ERROR)
```

```
    {
```

```
        syserr("close");
```

```
    }
```

```
if (sourcefd > 0)
```

```
{
```

```
    // 표준입력을 재정의한다.
```

```
    if (dup(sourcefd) != 0)
```

```
    {
```

```
        fatal("dup");
```

```
    }
```

```
}
```

// 소스파일을 연다.

```
else if (open(sourcefile, O_RDONLY, 0) == ERROR)
```

```
{
```

```

        fprintf(stderr, "Cannot open %s\n", sourcefile);
        exit(0);
    }
}

// 출력 파일 디스크립터가 있다면
if (destfd != 1)
{
    // 표준 출력을 닫는다.
    if (close(1) == ERROR)
    {
        syserr("close");
    }

    // 표준 출력을 재정의
    if (destfd > 1)
    {
        if (dup(destfd) != 1)
        {
            fatal("dup");
        }
    }
}

else
{
    // 파일이 없을 경우에 생성하고, 쓰기도 가능하게 파일을 설정
    flags = O_WRONLY | O_CREAT;

    // 추가 모드가 아닐 경우 파일을 비우는 옵션도 추가

```

```

    if (!append)
    {
        flags |= O_TRUNC;
    }

    // 파일을 연다.
    if (open(destfile, flags, 0666) == ERROR)
    {
        fprintf(stderr, "Cannot create %s\n", destfile);
        exit(0);
    }

    // 추가 모드라면 가장 끝을 찾는다.
    if (append)
    {
        if (lseek(1, 0L, 2) == ERROR) syserr("lseek");
    }
}

// 파일디스크립터를 닫는다.
for (fd =3; fd < MAXFD; fd++)
{
    close(fd);
}

return;
}

```

[실행결과]

```
>> echo "hello world" > result
>> echo "add test" >> result
>> cat result
hello world
add test
```

11-5. ls, pwd, cd, mkdir, rmdir, ln, cp, rm, mv, cat 명령을 구현하시오.

[소스코드 및 설명]

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include "common.h"
```

```
////////////////////////////////////
```

```
// 명령어를 파싱한다.
```

```
////////////////////////////////////
```

```
SYMBOL parse(int *waitpid, BOOLEAN makepipe, int *pipefdp)
```

```
{
```

```
    SYMBOL symbol, term;
```

```
    int argc, sourcefd, destfd;
```

```
    int pid, pipefd[2];
```

```
    char *argv[MAXARG+1], sourcefile[MAXFNAME];
```

```
    char destfile[MAXFNAME];
```

```
    char word[MAXWORD];
```

```
    BOOLEAN append;
```

```
    argc = 0;
```

```
    sourcefd = 0;
```

```
    destfd = 1;
```

```

while (TRUE)
{
    // 하나의 단어씩을 분석한다.
    switch (symbol = getsymbol(word))
    {
        // 일반 문자일 경우
        case S_WORD :
            if(argc == MAXARG)
            {
                fprintf(stderr, "Too many args.\n");
                break;
            }

            // 새로운 인자 배열을 메모리 할당 한다.
            argv[argc] = (char *) malloc(strlen(word)+1);

            if(argv[argc] == NULL)
            {
                fprintf(stderr, "Out of arg memory.\n");
                break;
            }

            // 명령어 복사
            strcpy(argv[argc], word);

            // arg 카운터 증가
            argc++;
            continue;

```

```

// < 일 경우
case S_LT :

    // 파이프가 열려있다면 오류
    if(makepipe)
    {
        fprintf(stderr, "Extra <.Wn");
        break;
    }

    // 소스파일의 심볼값을 검사한다.
    if(getsymbol(sourcefile) != S_WORD)
    {
        fprintf(stderr, "Illegal <.Wn");
        break;
    }

    sourcefd = BADFD;
    continue;

// > 혹은 >> 일 경우
case S_GT :
case S_GTGT :

    // 목적 파일이 정의되어있지 않을 경우 에러
    if(destfd != 1)
    {
        fprintf(stderr, "Extra > or >>.Wn");
        break;
    }

```

```

}

// 목적어 파일의 심볼타임이 문자열이 아니면 에러
if(getsymbol(destfile) != S_WORD)
{
    fprintf(stderr, "Illegal > or >>.Wn");
    break;
}

// GTGT일 경우 추가 모드로
destfd = BADFD;
append = (symbol == S_GTGT);
continue;

// |, &, :, 줄바꿈 문자일 경우 - 하나의 명령어 단위
case S_BAR :
case S_AMP :
case S_SEMI :
case S_NL :

    argv[argc] = NULL;
    // 심볼이 파이프() 일 경우
    if(symbol == S_BAR)
    {
        if(destfd != 1)
        {
            fprintf(stderr, "> or >> conflicts with |.Wn");
            break;
        }
    }

```



```

// 현재의 표준출력 디스크립터를 함께 넘겨 명령어를 분석한다.
term = parse(waitpid, TRUE, &destfd);
}

// 종료 문자 세팅
else
{
    term = symbol;
}

// 파이프가 열려있을 경우 파이프를 연결한다.
if (makepipe)
{
    if (pipe(pipefd) == ERROR)
    {
        syserr("pipe");
    }
    *pipefdp = pipefd[1];
    sourcefd = pipefd[0];
}

// 명령을 수행한다.
pid = execute(argc, argv, sourcefd, sourcefile,
               destfd, destfile, append, term == S_AMP);

// 파이프가 아닐 경우 기다릴 PID를 설정
if (symbol != S_BAR)
{

```

```

        *waitpid = pid;
    }

    // 인자값을 없을 경우
    if (argc == 0 && (symbol != S_NL || sourcefd > 1))
    {
        fprintf(stderr, "Missing command.\n");
    }

    // 인자로 입력된 값들의 메모리 해제
    while (--argc >= 0)
    {
        free(argv[argc]);
    }

    return term;

// 명령이 잘못되었을 경우 종료
case S_EOF :
    exit(0);
}
}
}

```

1)ls

[실행 결과]

```
jiwon@ubuntu:~/lab3$ ./minish
>> ls
fcntl.h command.c      typingttest      lab3test.c      1
filelock.c      target.c      execls  writefifo.c      file_copy
file_creat      blocksignal   t2.txt  formprocess.c   file_chdir
typingtest.c    selfpipe.c    file_creat.c    password.c      file_dir.c
daemonize.c     file_access.c iopipe  lab3-9.c         file_stat.c
file_link.c     blocksignal.c file_append.c   lab3-9  parser.o
selfpipe        lockdata1.c   writefifo       execute.c       ls
test2.txt       file_hole.c   file_rename     file_append     lab3-11.c
file_check.out  waitprocess   sendsignal      iopipe.c       lab3-11
raisesignal     simplesh     symbol.o        sendsignal.c    readfifo
file_chdir.c    pibonacci.c  file_dir        handlesignal.c  output.txt
waitprocess.c   command.o    file_io.c       pipetest       forkprocess
file_open.c     lab3-9client minish.c        viPreset       pibonacci
append.c        minish  file_umask      nonblockpipe    redirect.o
test1.txt       a.out  selectpipe     file_output     common.h
t.txt  lab3-6.c  pipetest.c     fifo  lockdata1
test.txt       selectpipe.c  testlock      tty1  file_output.c
daemonize      exitprocess   util.c  lab3-4. lab3-9server.c
lab3-9client.c blocksigusr.c test2  alarmsignal  lab3-4
readfifo.c     exitprocess.c atexitprocess.c lab3-4.c      Makefile
file_umask.c   nonblockpipe.c file_copy.c    ignoresignal.c file_io
atexitprocess  parser.c      file_check.c  blocksigusr   file_size.c
lab3-9server   redirect.c    ignoresignal  lockdata2     tty1.c
raisesignal.c  zombi  filelock      file_access   file_check
file_rename.c  typingtest   lockdata2.c   execute.o     ls.c
simplesh.c     file_link    symbol.c      zombi.c  handlesignal
file_stat      file_hole    lab3-6  util.o  file_size
execls.c       password     alarmsignal.c
```

2)pwd

[실행 결과]

```
>> pwd
/home/jiwon
>> cd lab3/
>> pwd
/home/jiwon/lab3
```

3)cd

[실행 결과]

```
>> pwd
/home/jiwon
>> cd lab3/
>> pwd
/home/jiwon/lab3
```

4)mkdir

[실행 결과]

```
>> mkdir new_test
>> ls
fcntl.h command.c      typingttest    lab3test.c     1
filelock.c             target.c       execls         writefifo.c    file_copy
file_creat             blocksignal    t2.txt         formprocess.c  file_chdir
typingtest.c           selfpipe.c     file_creat.c   password.c      file_dir.c
daemonize.c            file_access.c  iopipe         lab3-9.c        file_stat.c
file_link.c            blocksignal.c  file_append.c  lab3-9          parser.o
selfpipe               lockdata1.c    writefifo      execute.c       ls
test2.txt              file_hole.c    file_rename    file_append     lab3-11.c
file_check.out         waitprocess    sendsignal     iopipe.c        lab3-11
raisesignal           simplsh        symbol.o        new_test        sendsignal.c
readfifo              file_chdir.c   pibonacci.c    file_dir        handlesignal.c
output.txt             waitprocess.c  command.o      file_io.c       pipetest
forkprocess           file_open.c    lab3-9client   minish.c        viPreset
pibonacci             append.c       minish         file_umask      nonblockpipe
redirect.o            test1.txt      a.out          selectpipe      file_output
common.h              t.txt         lab3-6.c       pipetest.c      fifo
lockdata1             test.txt       selectpipe.c   testlock        tty1
file_output.c          daemonize      exitprocess    util.c          lab3-4.
lab3-9server.c         lab3-9client.c blocksigusr.c   test2           alarmsignal
lab3-4 readfifo.c       exitprocess.c  atexitprocess.c lab3-4.c
Makefile              file_umask.c   nonblockpipe.c file_copy.c      ignoresignal.c
file_io atexitprocess  parser.c       file_check.c     blocksigusr
file_size.c           lab3-9server   redirect.c     ignoresignal     lockdata2
tty1.c raisesignal.c     zombi         filelock         file_access
file_check            file_rename.c  typingtest     lockdata2.c      execute.o
ls.c simplsh.c         file_link      symbol.c         zombi.c
handlesignal          file_stat      file_hole      lab3-6          util.o
file_size             execls.c       password       alarmsignal.c
```

5)rmdir

[실행 결과]

```
>> rmdir new_test
>>
>>
>>
>> ls
fcntl.h command.c      typingttest    lab3test.c     1
filelock.c             target.c       execls         writefifo.c    file_copy
file_creat             blocksignal    t2.txt         formprocess.c  file_chdir
typingtest.c           selfpipe.c     file_creat.c   password.c      file_dir.c
daemonize.c            file_access.c  iopipe         lab3-9.c        file_stat.c
file_link.c            blocksignal.c  file_append.c  lab3-9          parser.o
selfpipe               lockdata1.c    writefifo      execute.c       ls
test2.txt              file_hole.c    file_rename    file_append     lab3-11.c
file_check.out         waitprocess    sendsignal     iopipe.c        lab3-11
raisesignal           simplsh        symbol.o        sendsignal.c    readfifo
file_chdir.c          pibonacci.c    file_dir       handlesignal.c  output.txt
waitprocess.c         command.o      file_io.c      pipetest        forkprocess
file_open.c           lab3-9client   minish.c       viPreset        pibonacci
append.c              minish         file_umask     nonblockpipe    redirect.o
test1.txt             a.out          selectpipe     file_output     common.h
t.txt lab3-6.c         pipetest.c     fifo            lockdata1
test.txt              selectpipe.c   testlock       tty1            file_output.c
daemonize             exitprocess    util.c         lab3-4.         lab3-9server.c
lab3-9client.c        blocksigusr.c  test2          alarmsignal     lab3-4
readfifo.c            atexitprocess.c atexitprocess.c lab3-4.c        Makefile
file_umask.c           nonblockpipe.c file_copy.c     ignoresignal.c  file_io
atexitprocess         parser.c       file_check.c   blocksigusr     file_size.c
lab3-9server          redirect.c     ignoresignal   lockdata2       tty1.c
raisesignal.c         zombi         filelock       file_access     file_check
file_rename.c         typingtest     lockdata2.c    execute.o       ls.c
simplsh.c             file_link      symbol.c       zombi.c         handlesignal
file_stat             file_hole      lab3-6         util.o          file_size
execls.c              password       alarmsignal.c
```

6)cp

[실행 결과]

```
>> cp happy.c copy.c
>> ls
fcntl.h command.c      typingttest    lab3test.c     1
filelock.c             target.c       execls         writefifo.c    file_copy
file_creat             blocksignal    t2.txt         formprocess.c  file_chdir
typingtest.c           selfpipe.c     file_creat.c   password.c     file_dir.c
daemonize.c            file_access.c  changtest.txt  iopipe         lab3-9.c
file_stat.c            file_link.c    copy.c         blocksignal.c  file_append.c
lab3-9 parser.o         result         selfpipe       lockdata1.c
writefifo              execute.c      ls             test2.txt      file_hole.c
file_rename            file_append    lab3-11.c      file_check.out waitprocess
sendsignal            iopipe.c      lab3-11 raisesignal simplish
symbol.o              sendsignal.c   readfifo       file_chdir.c   pibonacci.c
file_dir              handlesignal.c output.txt      waitprocess.c  command.o
file_io.c             pipetest      forkprocess    file_open.c    lab3-9client
minish.c              viPreset      pibonacci      append.c        minish
file_umask            nonblockpipe  redirect.o     a.out          selectpipe
file_output            common.h       t.txt          lab3-6.c        pipetest.c
fifo                  lockdata1     selectpipe.c   testlock       tty1
file_output.c          daemonize      exitprocess    util.c          lab3-4.
lab3-9server.c         lab3-9client.c blocksigusr.c  alarmsignal     lab3-4
readfifo.c            exitprocess.c  atexitprocess.c lab3-4.c        Makefile
file_umask.c          nonblockpipe.c file_copy.c    ignoresignal.c  file_io
atexitprocess         parser.c       file_check.c   blocksigusr     file_size.c
lab3-9server          redirect.c     ignoresignal   lockdata2       tty1.c
raisesignal.c         zombi         filelock       file_access     file_check
file_rename.c         typingtest     happy.c        lockdata2.c     execute.o
ls.c                  simplish.c    file_link      symbol.c         zombi.c
handlesignal          file_stat     file_hole      lab3-6          util.o
file_size             execls.c      password       alarmsignal.c
>> copy.c
Cannot execute copy.c
>> cat copy.c
hello~ test
```

7)rm

[실행 결과]

```
>> ls
fcntl.h command.c      typingttest    lab3test.c     1
filelock.c             target.c       execls         writefifo.c    file_copy
file_creat             blocksignal    t2.txt         formprocess.c  file_chdir
typingtest.c           selfpipe.c     file_creat.c   password.c     file_dir.c
daemonize.c            file_access.c  changtest.txt  iopipe         lab3-9.c
file_stat.c            file_link.c    copy.c         blocksignal.c  file_append.c
parser.o               result         selfpipe       lockdata1.c    writefifo
execute.c              ls             test2.txt      file_hole.c    file_rename
file_append            lab3-11.c      file_check.out waitprocess     sendsignal
iopipe.c              lab3-11 raisesignal simplish        symbol.o
sendsignal.c          readfifo       file_chdir.c   pibonacci.c    file_dir
handlesignal.c        output.txt     waitprocess.c  command.o      file_io.c
pipetest              forkprocess    file_open.c    lab3-9client   minish.c
viPreset              pibonacci      append.c        minish         file_umask
nonblockpipe          redirect.o     test1.txt      a.out          selectpipe
file_output            common.h       t.txt          lab3-6.c        pipetest.c
fifo                  lockdata1     selectpipe.c   testlock       tty1
file_output.c          daemonize      exitprocess    util.c          lab3-4.
lab3-9server.c         lab3-9client.c blocksigusr.c  test2          alarmsignal
lab3-4 readfifo.c     exitprocess.c  atexitprocess.c lab3-4.c
Makefile              file_umask.c   nonblockpipe.c file_copy.c     ignoresignal.c
file_io atexitprocess parser.c        file_check.c   blocksigusr
file_size.c           lab3-9server  redirect.c     ignoresignal   lockdata2
tty1.c raisesignal.c zombi         filelock       file_access
file_check            file_rename.c typingtest     lockdata2.c    execute.o
ls.c                  simplish.c    file_link      symbol.c        zombi.c
handlesignal          file_stat     file_hole      lab3-6          util.o
file_size             execls.c      password       alarmsignal.c
>> rm test1.txt
```

```
>> ls
fcntl.h command.c      typingttest  lab3test.c    1
filelock.c             target.c     execls        writefifo.c   file_copy
file_creat             blocksignal t2.txt        formprocess.c file_chdir
typingtest.c           selfpipe.c  file_creat.c  password.c    file_dir.c
daemonize.c            file_access.c changtest.txt iopipe        lab3-9.c
file_stat.c            file_link.c blocksignal.c file_append.c  lab3-9
parser.o               result      selfpipe      lockdata1.c   writefifo
execute.c              ls          test2.txt     file_hole.c   file_rename
file_append            lab3-11.c   file_check.out waitprocess    sendsignal
iopipe.c               lab3-11.c  raisesignal   simplesh      symbol.o
sendsignal.c           readfifo    file_chdir.c  pibonacci.c   file_dir
handlesignal.c         output.txt  waitprocess.c command.o      file_io.c
pipetest               forkprocess file_open.c   lab3-9client  minish.c
viPreset               pibonacci  append.c      minish        file_umask
nonblockpipe           redirect.o a.out         selectpipe    file_output
common.h               t.txt      lab3-6.c      pipetest.c    fifo
lockdata1              selectpipe.c testlock      tty1          file_output.c
daemonize               exitprocess util.c        lab3-4. lab3-9server.c
lab3-9client.c          blocksigusr test2          alarmsignal   lab3-4
readfifo.c              exitprocess.c atexitprocess.c lab3-4.c      Makefile
file_umask.c            nonblockpipe.c file_copy.c   ignoresignal.c file_io
atexitprocess           parser.c    file_check.c  blocksigusr   file_size.c
lab3-9server            redirect.c  ignoresignal lockdata2      tty1.c
raisesignal.c           zombi      filelock      file_access    file_check
file_rename.c           typingtest lockdata2.c   execute.o     ls.c
simplesh.c              file_link  symbol.c      zombi.c        handlesignal
file_stat               file_hole  lab3-6        util.o         file_size
execls.c                password   alarmsignal.c
```

8)mv

[실행 결과]

```
>> mv test.txt changtest.txt
>> ls
fcntl.h command.c      typingttest  lab3test.c    1
filelock.c             target.c     execls        writefifo.c   file_copy
file_creat             blocksignal t2.txt        formprocess.c file_chdir
typingtest.c           selfpipe.c  file_creat.c  password.c    file_dir.c
daemonize.c            file_access.c changtest.txt iopipe        lab3-9.c
file_stat.c            file_link.c blocksignal.c file_append.c  lab3-9
parser.o               result      selfpipe      lockdata1.c   writefifo
execute.c              ls          test2.txt     file_hole.c   file_rename
file_append            lab3-11.c   file_check.out waitprocess    sendsignal
iopipe.c               lab3-11.c  raisesignal   simplesh      symbol.o
sendsignal.c           readfifo    file_chdir.c  pibonacci.c   file_dir
handlesignal.c         output.txt  waitprocess.c command.o      file_io.c
pipetest               forkprocess file_open.c   lab3-9client  minish.c
viPreset               pibonacci  append.c      minish        file_umask
nonblockpipe           redirect.o test1.txt     a.out         selectpipe
file_output            common.h    t.txt        lab3-6.c      pipetest.c
fifo lockdata1         selectpipe.c testlock      tty1
file_output.c          daemonize  exitprocess   util.c        lab3-4.
lab3-9server.c         lab3-9client.c blocksigusr.c test2          alarmsignal
lab3-4 readfifo.c      exitprocess.c atexitprocess.c lab3-4.c
Makefile               file_umask.c nonblockpipe.c file_copy.c   ignoresignal.c
file_io atexitprocess parser.c        file_check.c  blocksigusr
file_size.c            lab3-9server redirect.c    ignoresignal lockdata2
tty1.c raisesignal.c zombi      filelock      file_access
file_check             file_rename.c typingtest   lockdata2.c   execute.o
ls.c simplesh.c        file_link   symbol.c      zombi.c
handlesignal           file_stat  file_hole    lab3-6        util.o
file_size              execls.c   password     alarmsignal.c
```

9)cat

[실행 결과]

```
>> echo "hello world" > result
>> echo "add test" >> result
>> cat result
hello world
add test
```

11-6. 파이프(|) 기능을 추가하도록 수정하시오.

[소스코드 및 설명]

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <common.h>
```

```
////////////////////////////////////
```

```
// 명령어를 파싱한다.
```

```
////////////////////////////////////
```

```
SYMBOL parse(int *waitpid, BOOLEAN makepipe, int *pipefdp)
```

```
{
```

```
    SYMBOL symbol, term;
```

```
    int argc, sourcefd, destfd;
```

```
    int pid, pipefd[2];
```

```
    char *argv[MAXARG+1], sourcefile[MAXFNAME];
```

```
    char destfile[MAXFNAME];
```

```
    char word[MAXWORD];
```

```
    BOOLEAN append;
```

```
    argc = 0;
```

```
    sourcefd = 0;
```

```
    destfd = 1;
```

```

while (TRUE)
{
    // 하나의 단어씩을 분석한다.
    switch (symbol = getsymbol(word))
    {
        // 일반 문자일 경우
        case S_WORD :
            if(argc == MAXARG)
            {
                fprintf(stderr, "Too many args.\n");
                break;
            }

            // 새로운 인자 배열을 메모리 할당 한다.
            argv[argc] = (char *) malloc(strlen(word)+1);

            if(argv[argc] == NULL)
            {
                fprintf(stderr, "Out of arg memory.\n");
                break;
            }

            // 명령어 복사
            strcpy(argv[argc], word);

            // arg 카운터 증가
            argc++;
            continue;

```



```

// < 일 경우
case S_LT :

    // 파이프가 열려있다면 오류
    if(makepipe)
    {
        fprintf(stderr, "Extra <.Wn");
        break;
    }

    // 소스파일의 심볼값을 검사한다.
    if(getsymbol(sourcefile) != S_WORD)
    {
        fprintf(stderr, "Illegal <.Wn");
        break;
    }

    sourcefd = BADFD;
    continue;

// > 혹은 >> 일 경우
case S_GT :
case S_GTGT :

    // 목적 파일이 정의되어있지 않을 경우 에러
    if(destfd != 1)
    {
        fprintf(stderr, "Extra > or >>.Wn");
    }

```

```

        break;
    }

    // 목적어 파일의 심볼타임이 문자열이 아니면 에러
    if(getsymbol(destfile) != S_WORD)
    {
        fprintf(stderr, "Illegal > or >>.Wn");
        break;
    }

    // GTGT일 경우 추가 모드로
    destfd = BADFD;
    append = (symbol == S_GTGT);
    continue;

    // |, &, :, 줄바꿈 문자일 경우 - 하나의 명령어 단위
    case S_BAR :
    case S_AMP :
    case S_SEMI :
    case S_NL :

        argv[argc] = NULL;
        // 심볼이 파이프() 일 경우
        if(symbol == S_BAR)
        {
            if(destfd != 1)
            {
                fprintf(stderr, "> or >> conflicts with |.Wn");
                break;
            }

```

```

    }

    // 현재의 표준출력 디스크립터를 함께 넘겨 명령어를 분석한다.
    term = parse(waitpid, TRUE, &destfd);
}

// 종료 문자 세팅
else
{
    term = symbol;
}

// 파이프가 열려있을 경우 파이프를 연결한다.
if (makepipe)
{
    if (pipe(pipefd) == ERROR)
    {
        syserr("pipe");
    }
    *pipefdp = pipefd[1];
    sourcefd = pipefd[0];
}

// 명령을 수행한다.
pid = execute(argc, argv, sourcefd, sourcefile,
               destfd, destfile, append, term == S_AMP);

// 파이프가 아닐 경우 기다릴 PID를 설정
if (symbol != S_BAR)

```

```

    {
        *waitpid = pid;
    }

    // 인자값을 없을 경우
    if (argc == 0 && (symbol != S_NL || sourcefd > 1))
    {
        fprintf(stderr, "Missing command.\n");
    }

    // 인자로 입력된 값들의 메모리 해제
    while (--argc >= 0)
    {
        free(argv[argc]);
    }

    return term;

    // 명령이 잘못되었을 경우 종료
    case S_EOF :
        exit(0);
    }
}
}

```

[실행 결과]

```
jiwon@ubuntu:~/lab3$ ./minish
>> ps aux | more
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.3 159836  7672 ?        Ss   07:57   0:08 /sbin/init auto noprompt
root         2  0.0  0.0      0     0 ?        S    07:57   0:00 [kthreadd]
root         4  0.0  0.0      0     0 ?        I<   07:57   0:00 [kworker/0:0H]
root         6  0.0  0.0      0     0 ?        I<   07:57   0:00 [mm_percpu_wq]
root         7  0.0  0.0      0     0 ?        S    07:57   0:01 [ksoftirqd/0]
root         8  0.0  0.0      0     0 ?        I    07:57   0:06 [rcu_sched]
root         9  0.0  0.0      0     0 ?        I    07:57   0:00 [rcu_bh]
root        10  0.0  0.0      0     0 ?        S    07:57   0:00 [migration/0]
root        11  0.0  0.0      0     0 ?        S    07:57   0:00 [watchdog/0]
root        12  0.0  0.0      0     0 ?        S    07:57   0:00 [cpuhp/0]
root        13  0.0  0.0      0     0 ?        S    07:57   0:00 [kdevtmpfs]
root        14  0.0  0.0      0     0 ?        I<   07:57   0:00 [netns]
root        15  0.0  0.0      0     0 ?        S    07:57   0:00 [rcu_tasks_kthre]
root        16  0.0  0.0      0     0 ?        S    07:57   0:00 [kauditd]
root        17  0.0  0.0      0     0 ?        S    07:57   0:00 [khungtaskd]
root        18  0.0  0.0      0     0 ?        S    07:57   0:00 [oom_reaper]
root        19  0.0  0.0      0     0 ?        I<   07:57   0:00 [writeback]
root        20  0.0  0.0      0     0 ?        S    07:57   0:00 [kcompactd0]
root        21  0.0  0.0      0     0 ?        SN   07:57   0:00 [ksmd]
root        22  0.0  0.0      0     0 ?        SN   07:57   0:00 [khugepaged]
root        23  0.0  0.0      0     0 ?        I<   07:57   0:00 [crypto]
root        24  0.0  0.0      0     0 ?        I<   07:57   0:00 [kintegrityd]
root        25  0.0  0.0      0     0 ?        I<   07:57   0:00 [kblockd]
root        26  0.0  0.0      0     0 ?        I<   07:57   0:00 [ata_sff]
root        27  0.0  0.0      0     0 ?        I<   07:57   0:00 [md]
root        28  0.0  0.0      0     0 ?        I<   07:57   0:00 [edac-poller]
root        29  0.0  0.0      0     0 ?        I<   07:57   0:00 [devfreq_wq]
root        30  0.0  0.0      0     0 ?        I<   07:57   0:00 [watchdogd]
root        34  0.0  0.0      0     0 ?        S    07:57   0:01 [kswapd0]
root        35  0.0  0.0      0     0 ?        S    07:57   0:00 [ecryptfs-kthrea]
root        77  0.0  0.0      0     0 ?        I<   07:57   0:00 [kthrotld]
root        78  0.0  0.0      0     0 ?        I<   07:57   0:00 [acpi_thermal_pm]
root        79  0.0  0.0      0     0 ?        S    07:57   0:00 [scsi_eh_0]
root        80  0.0  0.0      0     0 ?        I<   07:57   0:00 [scsi_tmf_0]
root        81  0.0  0.0      0     0 ?        S    07:57   0:00 [scsi_eh_1]
root        82  0.0  0.0      0     0 ?        I<   07:57   0:00 [scsi_tmf_1]
root        88  0.0  0.0      0     0 ?        I<   07:57   0:00 [ipv6_addrconf]
root        97  0.0  0.0      0     0 ?        I<   07:57   0:00 [kstrp]
root       114  0.0  0.0      0     0 ?        I<   07:57   0:00 [charger_manager]

root       124  0.0  0.0      0     0 ?        I    07:57   0:00 [kworker/0:2]
root       162  0.0  0.0      0     0 ?        I<   07:57   0:00 [mpt_poll_0]
root       163  0.0  0.0      0     0 ?        I<   07:57   0:00 [mpt/0]
root       164  0.0  0.0      0     0 ?        S    07:57   0:00 [scsi_eh_2]
root       165  0.0  0.0      0     0 ?        I<   07:57   0:00 [scsi_tmf_2]
--More--
[1]+  Stopped                  ./minish
```

5.2 프로젝트 완성도 평가

	완성도	우수성
프로그램 설계	80%	좋은 팀워크로 프로그램 설계 및 설계 구성요소 작성은 우수했다 생각합니다. 프로그램 개요 작성에 있어 어려움이 있었습니다. 각 기능 중 파이프 기능을 구현하는데 어려움이 있었습니다.
프로그램 개요 작성		
설계 구성 요소 작성	85%	
역할 분담	90%	
일정 관리 계획 작성	95%	
시그널 처리	80%	
파이프 기능	80%	
기본 명령어 및 파일처리 명령어	90%	
좀비프로세스	80%	
파일 재지향	85%	
셸 내 프로그램 종료 명령어	90%	
셸 프로그램 통합	85%	
셸 프로그램 구현	85%	
프로젝트 결과 보고서 작성	90%	

5.3 일정 계획 평가

단계	일정	달성도
프로그램 설계	18.11.12~18.11.14	100%
프로그램 개요 작성	18.11.12~18.11.14	100%
설계 구성 요소 작성	18.11.12~18.11.14	100%
역할 분담	18.11.12~18.11.14	100%
일정관리 계획 작성	18.11.13~18.11.13	100%
시그널 처리	18.11.14~18.11.15	75%
파이프 기능	18.11.14~18.11.16	80%
기본 명령어 및 파일처리 명령어	18.11.15~18.11.16	80%
좀비 프로세스	18.11.15~18.11.17	75%
파일 재지향	18.11.16~18.11.18	80%
셸 내 프로그램 종료 명령어	18.11.17~18.11.21	80%
셸 프로그램 통합	18.11.18~18.11.21	80%
셸 프로그램 구현	18.11.18~18.11.21	80%
프로젝트 결과 보고서	18.11.19~18.11.22	100%

5.4 역할 수행 평가

조원	역할	평가
곽지원	파일 재지향	85%
	파이프 기능	75%
	셸 프로그램 통합	85%
	보고서	80%
김예원	시그널 처리	80%
	기본 명령어 및 파일처리 명령어	85%
	셸 프로그램 구현	80%
	보고서	85%
박경인	좀비프로세스	80%
	파이프 기능	80%
	셸 프로그램 구현	80%
	보고서	85%
백소현	exit 명령어	85%
	기본 명령어 및 파일처리 명령어	90%
	셸 프로그램 통합	85%
	보고서	80%

6. 소감

곽지원 : 이번 과제를 통해 리눅스 환경에 더욱 익숙해 질 수 있었고 각 명령어들은 무슨 용도인지 검색도 해보고 ppt를 보고 공부를 하면서 확실하게 터득할 수 있었다. 리눅스 과제에 시간을 많이 투자했는데도 시간이 부족해서 팀 과제를 완벽하게 해내지 못한 것 같아서 아쉬움이 많이 남았다. 하지만 팀원들과 힘을합쳐 열심히 함께 했기 때문에 만족한다.

김예원 : 간단한 셸에 여러 기능을 구현함으로써 하나하나의 기능에 대해 더욱 구체적으로 공부할 수 있는 기회가 되었다. 각 팀원들이 기능을 분담하여 소스코드를 짰 뒤 한 프로그램에 병합하는 과정이 어려웠지만 하나하나 합쳐 가며 실행이 될 때 성취감을 느낄 수 있었다. 비록 셸의 모든 기능들을 구현하지 못했지만, 다음에는 더욱 효율적으로 셸 프로그램을 만들 수 있을 것 같다.

박경인 : 혼자서는 수행하기 어려웠을 이번 프로젝트를 조별과제를 통해 조원들과 협업하여 프로젝트를 끝까지 마무리하는 경험을 하였습니다. 파이프와 좀비프로세스에 대해서 지식이 부족했는데, 이번 과제를 통해서 파이프와 좀비프로세스에 대하여 좀더 심도깊은 공부를 하게 되어 뿌듯했습니다.

백소현 : 이번 과제를 수행하면서 리눅스의 전반적인 기능들에 대해서 실습과 함께 공부를 할 수 있어 좀 더 쉽게 받아 드릴 수 있었던 것 같다. 모든 기능을 다 구현 하지 못해 아쉬운 점이 많았다. C에 대한 지식도 좀 더 쌓아야 할 필요성이 있는 점도 있었던 것 같다. 전체적으로 정말 도움이 많이 되었던 과제들과 수업 시간이 었던 것 같아서 만족한다.기능들을 전부 구현하는 것에 의해서 좀 부족 했지만 팀원들의 협동으로 마지막까지 잘 해결한 것 같아서 좋았던 것 같다.